

Preliminaries

We introduce important cryptographic preliminaries here that includes cryptographic assumptions, algebraic operations, and important scheme definitions.

.0.3 Cyclic Group Operations. Let $\mathbb{G}$ be a finite cyclic group of prime order p . Let $G \in \mathbb{G}$ be a generator of $\mathbb{G}$, meaning that every element within the group can be expressed by applying the group operation to G repeatedly. A group $\mathbb{G}$ consists of an "additive" and a "scalar multiplication" operation:

- **Group Addition:** The addition of two elements $g_1, g_2 \in \mathbb{G}$ can be expressed by $g_1 + g_2 = g_3 \in \mathbb{G}$.
- **Scalar Multiplication:** Scalar multiplication of a group element $g \in \mathbb{G}$ by a scalar $a \in \mathbb{Z}_p$ is denoted as $a \cdot g$ (representing G added to itself a times).

.0.4 Bilinear Pairings. Let $\mathbb{G}_1$ and $\mathbb{G}_2$ be two cyclic groups of prime order p , with generators G_1 and G_2 , respectively. Let $\mathbb{G}_T$ be a cyclic target group of the same order p .

A bilinear pairing is an efficiently computable mapping $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ that satisfies the following properties:

- **Bi-linearity:** For all $a, b \in \mathbb{Z}_p$, $e(a \cdot_{\mathbb{G}_1} G_1, b \cdot_{\mathbb{G}_2} G_2) = e(G_1, G_2)^{ab}$. Where $\cdot_{\mathbb{G}_1}, \cdot_{\mathbb{G}_2}$ are scalar multiplication operations in group $\mathbb{G}_1, \mathbb{G}_2$ respectively and the exponentiation on the right hand side is the scalar multiplication operation in $\mathbb{G}_3$.
- **Non-degeneracy:** $e(G_1, G_2) \neq 1_{\mathbb{G}_T}$, where $1_{\mathbb{G}_T}$ is the identity element of $\mathbb{G}_T$.

.0.5 Cryptographic Assumptions.

DEFINITION 1 (RANDOM ORACLE MODEL). A Random Oracle $H : \mathcal{X} \rightarrow \mathcal{Y}$ functions as a publicly accessible black box executing a truly random function drawn uniformly from the space of all functions $\text{Func}(\mathcal{X}, \mathcal{Y})$.

Operationally, the Random Oracle maintains an initially empty state table L_H . Upon receiving a query $x \in \mathcal{X}$:

- (1) If a tuple (x, y) already exists in L_H , the oracle deterministically returns y .
- (2) If x has not been queried before, the oracle samples $y \leftarrow_{\$} \mathcal{Y}$ uniformly at random, adds (x, y) to L_H , and returns y .

DEFINITION 2 (PSEUDORANDOM FUNCTION (PRF)). Let $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ be a deterministic polynomial-time computable function, where $\mathcal{K}$ is the key space, $\mathcal{X}$ is the input domain, and $\mathcal{Y}$ is the output range.

We say that F is a secure Pseudorandom Function if, for $k \leftarrow_{\$} \mathcal{K}$ chosen uniformly at random, the function $F(k, \cdot)$ is computationally indistinguishable from a truly random function $f \leftarrow_{\$} \text{Func}(\mathcal{X}, \mathcal{Y})$.

Formally, the PRF security game is where Challenger samples a random bit $b \leftarrow_{\$} \{0, 1\}$. If $b = 0$, the Challenger initializes the real function $F(k, \cdot)$. If $b = 1$, the Challenger initializes a truly random function $f(\cdot)$. A Probabilistic Polynomial-Time (PPT) adversary A is granted oracle access to the chosen function and adaptively submits queries $x_i \in \mathcal{X}$. Eventually, A outputs a guess bit $b' \in \{0, 1\}$.

The PRF advantage of the adversary is defined as its ability to guess the Challenger's bit strictly better than random chance:

$$\text{Adv}_{A,F}^{\text{PRF}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|$$

F is a secure PRF if, for all PPT adversaries A , the advantage $\text{Adv}_{A,F}^{\text{PRF}}(\lambda)$ is bounded by a negligible function $\text{negl}(\lambda)$.

DEFINITION 3 (q -STRONG DIFFIE-HELLMAN (q -SDH) ASSUMPTION [19]). Let $\mathbb{G}_1$ be a cyclic group of prime order p with a randomly chosen generator $G_1 \in \mathbb{G}_1$. Let $x \leftarrow_{\$} \mathbb{Z}_p^*$ be a uniformly random secret scalar.

The one-generator q -Strong Diffie-Hellman problem is defined as follows: Given a $(q + 1)$ -tuple of group elements $(G_1, x \cdot G_1, x^2 \cdot G_1, \dots, x^q \cdot G_1) \in \mathbb{G}_1^{q+1}$, compute and output a pair:

$$\left(c, \frac{1}{x + c} \cdot G_1 \right) \in \mathbb{Z}_p \times \mathbb{G}_1$$

such that $c \in \mathbb{Z}_p \setminus \{-x\}$.

The q -SDH assumption states that for any Probabilistic Polynomial-Time (PPT) adversary A , the probability of A successfully outputting a valid pair $(c, \frac{1}{x+c} \cdot G_1)$ is strictly bounded by a negligible function in the security parameter λ :

$$\Pr \left[A(G_1, x \cdot G_1, \dots, x^q \cdot G_1) = \left(c, \frac{1}{x+c} \cdot G_1 \right) \right] \leq \text{negl}(\lambda)$$

DEFINITION 4 (ONE-MORE COMPUTATIONAL DIFFIE-HELLMAN (OM-CDH) ASSUMPTION [18]). Let $\mathbb{G}_1$ be a cyclic group of prime order p with generator G_1 . We define a security game where a PPT adversary A interacts with a Challenger providing two oracles: $\text{CHAL}(\cdot)$ and $\text{CDH}(\cdot)$.

- **CHAL() Oracle:** Upon each invocation, the oracle samples $x, y \leftarrow_{\$} \mathbb{Z}_p$, computes $X := x \cdot G_1$ and $Y := y \cdot G_1$, and appends the correct CDH solution $Z := (x \cdot y) \cdot G_1$ to a secret target list $\vec{Z}$. It returns the challenge pair (X, Y) to A .
- **CDH(X, Y) Oracle:** Upon invocation with any group elements (X, Y) , the oracle computes their discrete logarithms $x := \log_{G_1}(X)$ and $y := \log_{G_1}(Y)$, increments a query counter $q := q + 1$, and returns the exact CDH solution $(x \cdot y) \cdot G_1$.

At the conclusion of the game, A outputs a list of proposed solutions $\vec{V}$. A wins the game if and only if its output list perfectly matches the recorded challenge list ($\vec{V} = \vec{Z}$) AND the total number of queries q made to the CDH oracle is strictly less than the total number of challenges requested ($q < |\vec{Z}|$).

The OM-CDH assumption states that for any PPT adversary A , the probability of winning this game is negligible:

$$\text{Adv}_A^{\text{OM-CDH}}(\lambda) = \Pr \left[\vec{V} = \vec{Z} \wedge q < |\vec{Z}| \right] \leq \text{negl}(\lambda)$$

DEFINITION 5 (CO-COMPUTATIONAL DIFFIE-HELLMAN (CO-CDH) ASSUMPTION [6]). Let $\mathbb{G}_1$ and $\mathbb{G}_2$ be cyclic groups of prime order p equipped with a non-degenerate bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. Let G_1 and G_2 be randomly chosen generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively.

The co-Computational Diffie-Hellman problem is defined as follows: Given the tuple (G_1, G_2, X, H) where $X = x \cdot G_2 \in \mathbb{G}_2$ for a uniformly random scalar $x \leftarrow_{\$} \mathbb{Z}_p^*$, and $H \leftarrow_{\$} \mathbb{G}_1$ is a uniformly random curve point, compute the element $V = x \cdot H \in \mathbb{G}_1$.

The co-CDH assumption states that for all Probabilistic Polynomial-Time (PPT) adversaries A , the probability of successfully outputting V is strictly bounded by a negligible function in the security parameter λ :

$$\Pr \left[A(G_1, G_2, x \cdot G_2, H) = x \cdot H \right] \leq \text{negl}(\lambda)$$

Cryptographic Tools

We specify the cryptographic details of certain tools we used in our work.

DEFINITION 6 (BONEH-LYNN-SHACHAM (BLS) SIGNATURE SCHEME [6]). The BLS signature scheme operates over groups with bilinearity $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ with generators $G_1 \in \mathbb{G}_1$ and $G_2 \in \mathbb{G}_2$. It utilizes a cryptographic hash function $H_{\text{curve}} : \{0, 1\}^* \rightarrow \mathbb{G}_1$ modeled as a random oracle. The scheme consists of three polynomial-time algorithms:

- **KeyGen**(1^λ): Samples a uniformly random scalar $sk \leftarrow_{\$} \mathbb{Z}_p^*$ as the secret signing key, and computes the public verification key $pk = sk \cdot G_2 \in \mathbb{G}_2$.
- **Sign**(sk, m): To sign an arbitrary message $m \in \{0, 1\}^*$, the signer hashes the message to the curve to obtain a base point $M = H_{\text{curve}}(m) \in \mathbb{G}_1$. The signature is computed via scalar multiplication as $\sigma = sk \cdot M \in \mathbb{G}_1$.
- **Verify**(pk, m, σ): The verifier hashes the target message to obtain $M = H_{\text{curve}}(m)$ and accepts the signature σ if and only if the following bilinear pairing equation holds:

$$e(\sigma, G_2) = e(M, pk)$$

NOTE: BLS signature scheme is proven to be Existentially Unforgeable under Chosen Message Attacks (EUF-CMA) in the Random Oracle Model, provided the co-CDH assumption holds.

DEFINITION 7 (PolyCommit_{DL}: POLYNOMIAL COMMITMENT SCHEME (WEAKENED FOR NON-HIDING)). A d -polynomial commitment scheme enabling point evaluation consists of the following algorithms:

- **gen**(d) - A randomized algorithm that outputs a Structured Reference String SRS.
- **com**(f, SRS) - An algorithm that takes a polynomial $f \in \mathbb{F}_{<d}[X]$ and returns a constant-size commitment cm .
- **create_proof**(f, z, SRS) - An algorithm that evaluates the polynomial f at a specific point $z \in \mathbb{F}$ and outputs a witness proof π demonstrating the correctness of the evaluation.
- **verify_eval**($\text{cm}, z, s, \pi, \text{SRS}$) - A deterministic verification algorithm that takes a commitment cm , an evaluation point z , a claimed value s , and a witness proof π . It returns 1 (accept) if π is a valid proof that the underlying polynomial committed in cm evaluates to s at point z , and 0 (reject) otherwise.

Such that the scheme satisfies the following properties:

- **Completeness:** For a correctly generated SRS, polynomial $f \in \mathbb{F}_{<d}[X]$, and query point $z \in \mathbb{F}$, if $\text{cm} = \text{com}(f, \text{SRS})$, $s = f(z)$, and $\pi = \text{create_proof}(f, z, \text{SRS})$, then $\text{verify_eval}(\text{cm}, z, s, \pi, \text{SRS})$ outputs 1 with probability 1.
- **Evaluation Binding:** It is computationally infeasible for a malicious actor to produce proofs that open the same commitment to two different values at the exact same evaluation point. The probability that a probabilistic polynomial-time adversary can output a tuple $(\text{cm}, z, s, s', \pi, \pi')$ such that $s \neq s'$, but both $\text{verify_eval}(\text{cm}, z, s, \pi, \text{SRS}) = 1$ and $\text{verify_eval}(\text{cm}, z, s', \pi', \text{SRS}) = 1$, is strictly bounded by $\text{negl}(\lambda)$, where λ is the security parameter.

THEOREM 6 (KATE ET AL.[19], THEOREM 1). PolyCommit_{DL} is a secure polynomial commitment scheme provided the DL and q -Strong Diffie-Hellman (q -SDH) Assumption hold.

This theorem formally guarantees two critical properties for our system:

- (1) **Binding (t-SDH):** The S cannot convincingly open the KZG commitment to two different spatial blocks for the same index.
- (2) **Hiding (DL):** The C 's batched verification queries do not leak the underlying polynomial evaluations to an observing adversary, however, **we do not enforce this in our scheme as the OPRF-layer ensures this.**

DEFINITION 8 (2DH-OPRF EVALUATION PROTOCOL). The OPRF protocol is an interactive process between a Client and a Server:

- (1) **Client Blinding:** Client C samples a random blinding scalar $r \leftarrow \mathbb{Z}_p$, perform scalar multiplication on $g \in \mathbb{G}$, and transmits the blinded point to server S :

$$X \leftarrow r \cdot g$$

- (2) **Server Evaluation:** S evaluates the blinded point using its secret master key $\mathcal{K}_{\text{oprf}}$ (doing scalar multiplication) and returns the evaluated point to C :

$$Y \leftarrow \mathcal{K}_{\text{oprf}} \cdot X$$

- (3) **Client Un-blinding:** C removes the blinding scalar to recover the true evaluated curve point:

$$Z \leftarrow (1/r) \cdot Y = \mathcal{K}_{\text{oprf}} \cdot g$$

Security Proofs

We formally prove that our protocol is not only correct, but satisfies the security properties (outlined in section 3) under their respective security games.

THEOREM 7 (PROTOCOL COMPLETENESS). Let $\Pi_{\text{subset_vfy}}$ be the interactive subset authentication protocol. For any honestly generated global parameters pp and (sk, pk) , if S correctly registers dataset B , and C executes the Verify algorithm using a valid subset B' such that $b'_j = b_j$ for all $j \in I$, then the protocol outputs 1 with probability 1.

PROOF. To prove perfect completeness, we must demonstrate that the equation in Algorithm 7

$$e(C_{\text{agg}}, G_2) = e(P_{\text{tag,agg}}, pk) \cdot e(W_{\text{agg}}, [s]_2)$$

holds under honest execution.

Step 1: OPRF Tag Equivalence. Under honest execution ($b'_j = b_j$), the 2-pass OPRF trivially guarantees tag equivalence. The Client's blinding factor r_j and subsequent unblinding r_j^{-1} perfectly cancel over the Server's evaluation ($Y_j = K_{\text{oprf}} \cdot r_j \cdot M_j$). This ensures: $T'_j = T_j$.

Step 2: Keyed KZG Correctness for a Single Index. Next, we show that for any valid tag T_j , the Keyed KZG pairing equation holds. We first establish the correctness of the underlying unkeyed KZG scheme. Let $P(x)$ be the interpolated polynomial where $P(j) = T_j$. The quotient polynomial used to generate the witness is defined as $Q_j(x) = \frac{P(x) - T_j}{x - j}$, which gives the algebraic equivalence:

$$P(x) - T_j = Q_j(x)(x - j)$$

Evaluating this polynomial equation at the secret s yields $P(s) - T_j = Q_j(s)(s - j)$. When encoded as scalar multiplications on $\mathbb{G}_1$, the

unkeyed commitment is $C = P(s) \cdot G_1$ and the witness proof is $W_j = Q_j(s) \cdot G_1$. By the bilinear property of e , we have:

$$e(C - (T_j \cdot G_1), G_2) = e(W_j, s \cdot G_2 - j \cdot G_2)$$

Substituting $[s]_2 = s \cdot G_2$ and expanding the pairing properties, we can rewrite this as:

$$e(C, G_2) = e(T_j \cdot G_1, G_2) \cdot e(W_j, [s]_2) \cdot e(W_j, -j \cdot G_2)$$

Rearranging the terms to group the evaluation index j yields the standard rearranged KZG equation:

$$e(C + j \cdot W_j, G_2) = e(T_j \cdot G_1, G_2) \cdot e(W_j, [s]_2)$$

In our protocol, S multiplies the $\mathbb{G}_1$ points by sk . Pushing the scalar sk into the pairing arguments, we obtain:

$$e(sk \cdot C + sk \cdot j \cdot W_j, G_2) = e(T_j \cdot G_1, sk \cdot G_2) \cdot e(sk \cdot W_j, [s]_2)$$

Substituting our defined protocol variables $C_{sk} = sk \cdot C$, $\pi_{sk,j} = sk \cdot W_j$, and the public key $pk = sk \cdot G_2$, the single-point Keyed KZG equation correctly holds:

$$e(C_{sk} + j \cdot \pi_{sk,j}, G_2) = e(T_j \cdot G_1, pk) \cdot e(\pi_{sk,j}, [s]_2) \quad (1)$$

Step 3: Batched Folding Correctness. Finally, we prove that our randomized folding technique employed in Batch_Verify_Eval preserves the equality of Equation 1.

The Client has the padded index set I_{pad} . For authentic queries ($j \in I$), the Client samples random scalars $\gamma_j \leftarrow \mathbb{Z}_p^*$. For dummy queries ($j \notin I$), the Client assigns a zero-weight folding challenge $\gamma_j = 0$.

Multiplying Equation 1 by γ_j implies exponentiation the pairing results. Moving the scalar γ_j inside the $\mathbb{G}_1$ arguments of the pairings yields:

$$e(\gamma_j \cdot C_{sk} + \gamma_j \cdot j \cdot \pi_{sk,j}, G_2) = e(\gamma_j \cdot T_j \cdot G_1, pk) \cdot e(\gamma_j \cdot \pi_{sk,j}, [s]_2)$$

Taking the product over the entire padded set $j \in I_{\text{pad}}$ and aggregating:

$$\begin{aligned} e\left(\sum_{j \in I_{\text{pad}}} \gamma_j \cdot C_{sk} + \sum_{j \in I_{\text{pad}}} \gamma_j \cdot j \cdot \pi_{sk,j}, G_2\right) &= \\ e\left(\sum_{j \in I_{\text{pad}}} \gamma_j \cdot T_j \cdot G_1, pk\right) \cdot e\left(\sum_{j \in I_{\text{pad}}} \gamma_j \cdot \pi_{sk,j}, [s]_2\right) \end{aligned}$$

Crucially, because scalar multiplication by zero yields the point at infinity ($\mathbb{Z}_1$), all terms where $j \notin I$ mathematically vanish from the summation. The aggregation resolves exclusively to the authentic subset I , despite the Server processing N_{pad} queries.

Substituting the variables computed in Algorithm 7:

$$\begin{aligned} C_{\text{agg}} &= \left(\sum_{j \in I_{\text{pad}}} \gamma_j\right) \cdot C_{sk} + \sum_{j \in I_{\text{pad}}} (\gamma_j \cdot j) \cdot \pi_{sk,j} \\ P_{\text{tag,agg}} &= \left(\sum_{j \in I_{\text{pad}}} \gamma_j \cdot T_j\right) \cdot G_1 \\ W_{\text{agg}} &= \sum_{j \in I_{\text{pad}}} \gamma_j \cdot \pi_{sk,j} \end{aligned}$$

The resulting equation $e(C_{\text{agg}}, G_2) = e(P_{\text{tag,agg}}, pk) \cdot e(W_{\text{agg}}, [s]_2)$ allows the client to perform batch verification. The algorithm returns 1 when this equation holds, thus proving completeness even under padded conditions. $\square$

1 Security Proofs

Game 1: Index Extraction (IDX_EXT)

- (1) **Setup:** The Challenger generates pp by running the Setup algorithm and hand to a semi-malicious Adversary A.
- (2) **Challenge:** A outputs two distinct candidate index sets I_0 and I_1 of equal size $|I| \leq N_{\text{pad}}$. The Challenger uniformly samples a bit $\beta \leftarrow \{0, 1\}$, constructs the padded, shuffled, and blinded query vector $\vec{X}$ of size N_{pad} using I_β , and sends $\vec{X}$ to A.
- (3) **Evaluation & Corruption:** A evaluates the vector but may intentionally return k malformed responses to mount a selective failure attack. A returns the evaluated vector $\vec{Z}$ to the Challenger.
- (4) **Observation:** The Challenger unblinds and verifies. If any of the k malformed responses corresponds to index $j \in I_\beta$, the Challenger aborts and outputs $\perp$. Otherwise, it outputs $\top$. A is given this status flag.
- (5) **Guess:** A outputs a guess $\beta' \in \{0, 1\}$.

The Adversary's advantage is defined as $\text{Adv}_A^{\text{IDX_EXT}}(\lambda) = |\Pr[\beta' = \beta] - \frac{1}{2}|$.

THEOREM 8. *A semi-malicious adversary A executing a selective failure attack with k corruptions, the per-iteration advantage is bounded by $\text{Adv}_A^{\text{IDX_EXT}}(\lambda) \leq \frac{1}{N_{\text{pad}}}$ under the optimal strategy ($k = 1$).*

PROOF. To prove this bound, we analyze two cases: a passive execution ($k = 0$) and an active selective failure execution ($k \geq 1$).

Case 1: Passive Execution ($k = 0$). If the Server correctly evaluates all queries, the Challenger always outputs $\top$. The adversary's view consists solely of the query vector $\vec{X}$. This is the honest-but-curious case.

During blinding, we have the blinding factor $r_j \leftarrow \mathbb{Z}_p^*$. It serves as a perfect one-time pad in the prime-order group $\mathbb{G}_1$, rendering every point uniformly distributed. Hence, $X_j = r_j \cdot H_{\text{curve}}(j \parallel b'_j)$ is indistinguishable from dummy queries $X_j = r_j \cdot M'_j$. Furthermore, the $\vec{X}$ is randomly shuffled. Thus, the distributions of $\vec{X}$ under I_0 and I_1 are perfectly identical, yielding $\Pr[\beta' = \beta] = \frac{1}{2}$ and an advantage of 0.

Case 2: Selective Failure ($k \geq 1$). The adversary corrupts exactly k ciphertexts to observe the Challenger's abort status. Let E_{abort} denote the event that the Challenger outputs $\perp$. The Challenger aborts if and only if at least one corrupted ciphertext maps to an authentic index in I_β .

Because the vector is randomly shuffled, the k corruptions land on authentic indices following a hypergeometric distribution. The

probability of NOT aborting (all k corruptions hit dummy padding) is:

$$\Pr[\neg E_{\text{abort}}] = \frac{\binom{N_{\text{pad}} - |I|}{k}}{\binom{N_{\text{pad}}}{k}}$$

Therefore, the probability an abort is $\Pr[E_{\text{abort}}] = 1 - \Pr[\neg E_{\text{abort}}]$.

If the Challenger does not abort, A learns that the chosen positions were dummies. If the Challenger aborts, A learns that at least one corrupted position is an authentic index.

To maximize extraction precision, the optimal strategy for A is to set $k = 1$ where the abort probability simplifies to:

$$\Pr[E_{\text{abort}} \mid k = 1] = \frac{|I|}{N_{\text{pad}}}$$

Conditioned on $[E_{\text{abort}} \mid k = 1]$ and call the corrupted index is x , the probability that x corresponds to the specific distinguishing index between I_0 and I_1 is uniformly distributed across the subset size:

$$\Pr[\beta' = \beta \mid [E_{\text{abort}} \mid k = 1]] = \frac{1}{|I|}$$

By the Law of Total Probability, the maximum distinguishing advantage gained in a single iteration is:

$$\begin{aligned} \text{Adv}_{\text{A}}^{\text{IDX_EXT}}(\lambda) &= \Pr[E_{\text{abort}} \mid k = 1] \times \Pr[\beta' = \beta \mid E_{\text{abort}}] \\ &= \left(\frac{|I|}{N_{\text{pad}}} \right) \times \left(\frac{1}{|I|} \right) \\ &= \frac{1}{N_{\text{pad}}} \end{aligned}$$

□

Game 2: Offline Data Extraction (DTA_EXT_{offline})

- (1) **Setup:** The Challenger runs Setup and Registration on a dataset B drawn from a known dictionary $\mathcal{DB}$. The adversary A is given pp , the commitment C_{sk} , and the complete proof set $\{\pi_{sk}\}$.
- (2) **Knowledge Phase:** A can query a set of valid scalar tags $\{T_j\}_{j \in I_{\text{known}}}$, where the queried index set $I_{\text{known}} \subset \{1, \dots, N\}$ is strictly bounded by $|I_{\text{known}}| \leq Q \in \mathbb{N}$.
- (3) **Output:** C outputs a block b^* and an index j^* such that $j^* \notin I_{\text{known}}$. C wins if b^* is the correct block at index j^* in B .

The Client's offline advantage is defined as $\text{Adv}_{\text{A}}^{\text{DTA_EXT}_{\text{offline}}}(\lambda) = \Pr[m^* = m_{j^*} \wedge j^* \notin I_{\text{known}}]$.

THEOREM 9. *Under the One-More Computational Diffie-Hellman (OM-CDH) assumption in the Random Oracle Model, the offline data extraction advantage $\text{Adv}_{\text{A}}^{\text{DTA_EXT}_{\text{offline}}}(\lambda)$ is negligible.*

PROOF. Suppose there exists a PPT adversary A that breaks the DTA_EXT_{offline} game with non-negligible advantage ϵ . We construct a simulator SIM that uses A as a subroutine to solve the OM-CDH problem. To do this proof, it is important that we can successfully simulate the A's view for the whole DTA_EXT_{offline} game. In our setting, the SIM controls the two random oracles H_{curve} and H_{field} .

Setup & Parameter Generation: SIM receives an OM-CDH challenge pair $(A, B) \in \mathbb{G}_1^2$ from its Challenger, where $A = a \cdot G_1$

and $B = b \cdot G_1$. SIM's objective is to compute the CDH solution $V^* = (a \cdot b) \cdot G_1$.

SIM treats $A = \mathcal{K}_{\text{opr}} \cdot G_1 \implies \mathcal{K}_{\text{opr}} = a$ and provides A with the public parameters pp, pk (by executing the Setup(1^λ) algorithm). SIM now obtains $sk \leftarrow_{\$} \mathbb{Z}_p$, it samples a mock dataset $DS = \{b_1, \dots, b_N\}$ from the dictionary $\mathcal{DB}$. Finally SIM randomly guesses the target index and block (j^*, b^*) that A will attempt to forge, where $j^* \notin I_{\text{known}}$ and $b^* \in \mathcal{DB}$.

Random Oracle Simulation: SIM fully controls the random oracles H_{curve} and H_{field} and maintains corresponding query lists L_{curve} and L_{field} .

To generate the commitment C_{sk} and the set of proofs $\{\pi_{sk}\}$, SIM must first construct a complete tag set for each data block $\mathcal{T} = \{T_1, \dots, T_N\}$. It does so as follows:

- **For all $j \neq j^*$ (wrong guess):** SIM samples $r_j \leftarrow_{\$} \mathbb{Z}_p^*$, programs $H_{\text{curve}}(j \parallel b_j) := r_j \cdot G_1$, and computes the evaluated point $V_j = r_j \cdot A$. It then queries its own H_{field} simulation to obtain the true tag $T_j = H_{\text{field}}(V_j)$.
- **For the target j^* (correct guess):** SIM programs the R.O output to be the second OM-CDH challenge $H_{\text{curve}}(j^* \parallel b^*) = B$. The corresponding evaluated point is the unknown CDH solution $V^* = a \cdot B$. To bypass this unknown value, SIM samples a random scalar $T^* \leftarrow_{\$} \mathbb{Z}_p$ and temporarily assigns it as the tag for index j^* . SIM implicitly defines $H_{\text{field}}((a \cdot b) \cdot G_1) := T^*$.

Possessing the complete tag set $\mathcal{T}$, SIM executes the honest registration algorithm Gen_Commitment($pp, sk, \mathcal{T}$) to produce the commitment C_{sk} and the complete proof set $\{\pi_{sk}\}$.

Knowledge Phase Simulation: SIM must supply A with a set of Q valid scalar tags corresponding to a queried index subset I_{known} . SIM defines I_{known} as a subset of size Q drawn uniformly from $\{1, \dots, N\} \setminus \{j^*\}$. For each $j \in I_{\text{known}}$, SIM must provide the correct evaluation tag without possessing the master key a . Because SIM previously programmed the both random oracles for all $j \neq j^*$ using known scalars r_j such that $H_{\text{curve}}(j \parallel b_j) = r_j \cdot G_1$, SIM computes the valid evaluated points entirely from public values and the OM-CDH challenge element A as $V_j = r_j \cdot A$. SIM passes each V_j into its H_{field} simulation to retrieve the valid scalar tags T_j , and provides the resulting set $\{T_j\}_{j \in I_{\text{known}}}$ to A.

Indistinguishability of Simulation: We argue that A's view in this simulated environment is perfectly indistinguishable from the real DTA_EXT_{offline} game.

- (1) **Parameters and Keys:** The public parameters pp and keys (sk, pk) are generated using the exact honest algorithms. The simulated OPRF key is implicitly defined by the OM-CDH challenge $A = a \cdot G_1$. Because $a \leftarrow_{\$} \mathbb{Z}_p^*$ in the OM-CDH game, A is uniformly distributed in $\mathbb{G}_1$, perfectly matching the distribution of a real $\mathcal{K}_{\text{opr}}$.
- (2) **Random Oracles:** In the real game, H_{curve} outputs uniformly random curve points. In the simulation, both $r_j \cdot G_1$ (since $r_j \leftarrow_{\$} \mathbb{Z}_p^*$) and the OM-CDH challenge $B = b \cdot G_1$ are uniformly distributed generators in $\mathbb{G}_1$. Similarly, all outputs of H_{field} , including the temporarily assigned scalar T^* , are uniformly distributed over $\mathbb{Z}_p$, perfectly mirroring a true random oracle.

- (3) **Commitments and Proofs:** Because the underlying scalar tag set $\mathcal{T}$ is perfectly distributed, feeding it into the honest Gen_Commitment algorithm guarantees that the commitment C_{sk} and the proofs $\{\pi_{sk}\}$ are mathematically identical to those produced by an honest Server in the real world.

Because the simulated view is perfectly identical to the real game, A's probability of successfully outputting a valid forgery remains exactly ϵ .

Execution and Extraction: SIM invokes the adversary A, providing it with $pp, pk, C_{sk}, \{\pi_{sk}\}$, and the simulated tags $\{T_j\}_{j \in I_{\text{known}}}$.

Using these inputs, A executes its offline dictionary attack and eventually outputs its forgery attempt: a target index $j^\dagger \notin I_{\text{known}}$ and a block guess $m^\dagger \in \mathcal{D}$.

If $(j^\dagger, b^\dagger) \neq (j^*, b^*)$, SIM's initial guess was incorrect, and SIM immediately aborts the simulation.

However, if the guess is correct ($(j^\dagger, b^\dagger) = (j^*, b^*)$), SIM proceeds with extraction. For A to win the game, it must have correctly computed the valid tag T^* corresponding to the dictionary block b^* . Because H_{field} is modeled as a random oracle, A cannot predict T^* without querying H_{field} on the exact pre-image $V^* = K_{\text{opr}} \cdot H_{\text{curve}}(j^* \parallel b^*)$. Thus, if A succeeds, the pre-image V^* must exist within SIM's recorded query list L_{field} with overwhelming probability.

Because SIM strictly programmed $H_{\text{curve}}(j^* \parallel b^*) = B$ during the setup phase, the required pre-image inside the random oracle is exactly $V^* = K_{\text{opr}} \cdot B = (a \cdot b) \cdot G_1$.

To finalize the reduction, SIM simply iterates through all points V recorded in L_{field} and outputs them as a batch solution to the OM-CDH Challenger and one of them contains the correct answer to the OM-CDH challenge.

The probability that SIM correctly guesses (j^*, b^*) is exactly $1/(N \cdot |\mathcal{D}|)$. Therefore, SIM's total advantage is:

$$\text{Adv}_{\text{SIM}}^{\text{OM-CDH}}(\lambda) \geq \frac{1}{N \cdot |\mathcal{D}|} \cdot \text{Adv}_A^{\text{DTA_EXT}_{\text{offline}}}(\lambda) - \frac{q_H}{p}$$

where q_H is the maximum number of H_{field} queries made by A. Assuming OM-CDH is hard, $\text{Adv}_{\text{SIM}}^{\text{OM-CDH}}(\lambda)$ must be negligible. Because N and $|\mathcal{D}|$ are polynomially bounded, A's offline data extraction advantage $\text{Adv}_A^{\text{DTA_EXT}_{\text{offline}}}(\lambda)$ must also be negligible, concluding the proof. $\square$

Online Data Extraction Bound (DTA_EXT_{online})

In the following corollary, we show that the online data extraction advantage of the adversary is tightly bounded by the protocol's batch size and session limit.

COROLLARY 1. *Let Q be the upper bound on the number of interactive Verify protocol executions permitted by the Server. Assuming the conditions of Theorem 9 hold, the online data extraction advantage of any adversary A is bounded by:*

$$\text{Adv}_A^{\text{DTA_EXT}_{\text{online}}}(\lambda) \leq \frac{Q \cdot N_{\text{pad}}}{|\mathcal{D}|} + \text{negl}(\lambda)$$

PROOF. Because Theorem 9 guarantees that the adversary cannot verify block guesses locally without the Server's master evaluation key K_{opr} , the adversary is mathematically forced to submit its guesses to the honest Server via the Verify protocol.

Each protocol execution acts as an oracle query that returns Accept/Reject for a submitted batch of blocks. As the protocol requires a batch size of N_{pad} , a malicious Client can fill all N_{pad} blinded query slots with distinct candidate blocks from the known dictionary $\mathcal{DB}$.

Because the Server enforces Q total authentication sessions, the adversary can test $Q \cdot N_{\text{pad}}$ distinct candidate blocks. Assuming the target block is drawn uniformly from the dictionary, the probability of the adversary successfully querying the correct block across all permitted sessions is exactly $\frac{Q \cdot N_{\text{pad}}}{|\mathcal{D}|}$, plus the negligible probability that the adversary breaks the OM-CDH assumption. $\square$

Game 3: Data Forgery (DTA_FAB)

- (1) **Setup:** The Challenger runs $\text{Setup}(\lambda) \rightarrow (pp, sk, pk)$. For a dataset $B = \{b_1, \dots, b_N\}$, the Challenger executes the honest registration algorithm to generate the OPRF key $\mathcal{K}_{\text{opr}}$, the authentic scalar tags $\{T_1, \dots, T_N\}$, the commitment C_{sk} , and the complete proof set $\{\pi_{sk}\}$. A is given $(pp, pk, B, C_{sk}, \{T_j\}, \{\pi_{sk}\})$.
- (2) **Output:** A outputs a target spatial index $j^* \in \{1, \dots, N\}$, a fabricated block guess b^* , a forged scalar tag T^* , and a forged proof π^* . The target scalar tag is defined by the protocol's honest evaluation linkage: $T^* = H_{\text{field}}(\mathcal{K}_{\text{opr}} \cdot H_{\text{curve}}(j^* \parallel b^*))$.

The adversary wins if the Keyed KZG pairing equation accepts the forged proof. The advantage is defined as:

$$\text{Adv}_A^{\text{DTA_FAB}}(\lambda) = \Pr \left[\text{verify_eval}(pp, C_{sk}, j^*, T^*, \pi^*) = 1 \wedge b^* \neq b_{j^*} \right]$$

THEOREM 10. *Under the q -Strong Diffie-Hellman (q -SDH) assumption and assuming H_{curve} and H_{field} are collision-resistant random oracles, $\text{Adv}_A^{\text{DTA_FAB}}(\lambda)$ is negligible.*

PROOF. Suppose a PPT adversary A wins the DTA_FAB game. By definition, A outputs a tuple (j^*, b^*, T^*, π^*) such that the fabricated block $b^* \neq b_{j^*}$ and the verify_eval equation holds:

$$e(C_{sk}, G_2) = e(T^* \cdot G_1, pk) \cdot e(\pi^*, [s]_2 - j^* \cdot G_2)$$

1: Random Oracle Linkage (Preventing Mix-and-Match).

A trivial forgery attempt might involve A attempting to reuse an authentic tag T_{j^*} for a different block b^* . Because H_{curve} and H_{field} are modeled as collision-resistant random oracles, any change to the input pre-image $(j^* \parallel b^*)$ yields a completely independent and unpredictable scalar tag.

Since $b^* \neq b_{j^*}$, the collision resistance of the random oracles mathematically guarantees that $H_{\text{curve}}(j^* \parallel b^*) \neq H_{\text{curve}}(j^* \parallel b_{j^*})$, which in turn guarantees $T^* \neq T_{j^*}$ with overwhelming probability.

2: Reduction to Evaluation Binding.

First, observe that because H_{curve} and H_{field} are modeled as random oracles, finding an $b^* \neq b_{j^*}$ such that their resulting scalar tags collide ($T^* = T_{j^*}$) is infeasible. Thus, A's output implies $T^* \neq T_{j^*}$.

Next, we reduce the Keyed KZG verification to the standard KZG polynomial commitment scheme's verification. By the definition

of our setup and registration algorithms, the commitment is $C_{sk} = sk \cdot C$ and the public key is $pk = sk \cdot G_2$, where C is the standard unkeyed polynomial commitment to tags $\{T_1, \dots, T_N\}$, and $sk \in \mathbb{Z}_p^*$ is the non-zero secret key. We can rewrite A's successful pairing equation as:

$$e(sk \cdot C, G_2) = e(T^* \cdot G_1, sk \cdot G_2) \cdot e(\pi^*, [s]_2 - j^* \cdot G_2)$$

By leveraging the bilinearity of the pairing function to factor out the scalar sk , we obtain:

$$e(C, G_2)^{sk} = e(T^* \cdot G_1, G_2)^{sk} \cdot e((sk)^{-1} \cdot \pi^*, [s]_2 - j^* \cdot G_2)^{sk}$$

Taking the sk -th root of both sides (which is well-defined since $sk \neq 0$), we arrive at the standard unkeyed KZG verification equation:

$$e(C, G_2) = e(T^* \cdot G_1, G_2) \cdot e(W^*, [s]_2 - j^* \cdot G_2)$$

where $W^* = (sk)^{-1} \cdot \pi^* \in \mathbb{G}_1$.

In the Setup phase of the DTA_FAB game, the Challenger provided A with the authentic proof set, which includes π_{sk, j^*} for the authentic tag T_{j^*} . Factoring out sk from this authentic proof yields $W_{j^*} = (sk)^{-1} \cdot \pi_{sk, j^*}$.

Consequently, A has effectively produced two valid, conflicting standard KZG witnesses (W_{j^*}, W^*) for two different evaluations (T_{j^*}, T^*) at the exact same index j^* for the same commitment C .

In the formal security analysis of the KZG polynomial commitment scheme [19], this scenario exactly violates the *Evaluation Binding* property (Definition 5). Theorem 1 of Kate, Zaverucha, and Goldberg strictly proves that any PPT adversary capable of breaking Evaluation Binding can be used to construct a reduction that solves the q -Strong Diffie-Hellman (q -SDH) problem.

Therefore, any adversary that wins our DTA_FAB game directly violates the Evaluation Binding of the underlying polynomial commitment scheme. Assuming q -SDH is hard, the advantage $\text{Adv}_A^{\text{DTA_FAB}}(\lambda)$ must be strictly negligible. $\square$

Game 4: Existential Signature Forgery (EUF_CMA)

- (1) **Setup:** The Challenger generates the public parameters pp and keys (sk, pk) , where $sk \leftarrow_{\$} \mathbb{Z}_p^*$ and $pk = sk \cdot G_2 \in \mathbb{G}_2$. A is given (pp, pk) .
- (2) **Queries:** A is granted adaptive access to a Registration Oracle $\text{Register}(sk, \cdot)$. A submits polynomially many chosen datasets $\mathcal{B}_i$. The oracle returns the valid Keyed Commitment COM_{sk}^i . The Challenger maintains a list of all queried datasets $\mathcal{Q} = \{\mathcal{B}_1, \dots, \mathcal{B}_q\}$.
- (3) **Output:** A outputs a target dataset $\mathcal{B}^*$ and a forged commitment COM^* .

The adversary wins if the forged commitment evaluates correctly for the target dataset using the public key, but the dataset was never registered. The advantage is defined as:

$$\text{Adv}_A^{\text{EUF_CMA}}(\lambda) = \Pr \left[\text{Register}(sk, \mathcal{B}^*) = \text{COM}^* \wedge \mathcal{B}^* \notin \mathcal{Q} \right]$$

THEOREM 11. *Under the co-Computational Diffie-Hellman (co-CDH) assumption and the q -Strong Diffie-Hellman (q -SDH) assumption, the existential forgery advantage $\text{Adv}_A^{\text{EUF_CMA}}(\lambda)$ is negligible.*

PROOF. Suppose a Probabilistic Polynomial-Time (PPT) adversary A wins the EUF_CMA game with a non-negligible advantage ϵ . By definition, A outputs a target dataset $\mathcal{B}^*$ and a COM^* such that $\mathcal{B}^* \notin \mathcal{Q}$ and $\text{Register}(sk, \mathcal{B}^*) = \text{COM}^*$.

By the Register algorithm (Algorithm 2), the commitment ($\text{COM}^* = sk \cdot C^*$), where C^* is the non-keyed polynomial commitment, must satisfy the following equation:

$$e(\text{COM}^*, G_2) = e(C^*, sk \cdot G_2) = e(C^*, pk)$$

We now construct a strict reduction to the Existential Unforgeability under Chosen Message Attacks (EUF-CMA) of the Boneh-Lynn-Shacham (BLS) signature scheme [6]. We build a simulator SIM that uses A to forge a BLS signature.

Setup: SIM receives a BLS public key $pk_{BLS} = sk_{BLS} \cdot G_2 \in \mathbb{G}_2$ and is granted access to a BLS signing oracle $\mathcal{O}_{\text{Sign}}(sk_{BLS}, \cdot)$. SIM executes Setup algorithm (Alg 1) to generate the public parameters pp , samples a OPRF key $\mathcal{K}_{\text{opr}} \leftarrow_{\$} \mathbb{Z}_p^*$, and sets the public key $pk := pk_{BLS}$. SIM provides (pp, pk) to A.

Queries: When A queries the Registration Oracle $\text{Register}(sk, \cdot)$ with a chosen dataset $\mathcal{B}_i$, SIM uses $\mathcal{K}_{\text{opr}}$ to compute the tags and construct the unkeyed KZG commitment $C_i \in \mathbb{G}_1$. To apply the unknown sk , SIM submits C_i (acting as the message digest) to its BLS signing oracle, receiving the signature $\sigma_i = sk_{BLS} \cdot C_i$. SIM returns σ_i to A as the valid Keyed Commitment COM_{sk}^i .

Extraction: A outputs its valid forgery $(\mathcal{B}^*, \text{COM}^*)$ and hands it back to SIM. To evaluate the signature, SIM needs to reconstruct the underlying unkeyed KZG commitment C^* corresponding to $\mathcal{B}^*$. Because SIM possesses OPRF key $\mathcal{K}_{\text{opr}}$, it iterates through $\mathcal{B}^*$ and computes the authentic scalar tag for each block exactly as an honest Server would: $T_j^* = H_{\text{field}}(K_{\text{opr}} \cdot H_{\text{curve}}(j \parallel b_j^*))$. Using this complete, authenticated tag set and the trusted setup parameters within pp , SIM interpolates the dataset polynomial and computes the standard unkeyed KZG commitment $C^* \in \mathbb{G}_1$. This C^* serves as the exact mathematical message digest that A implicitly signed.

Because A won the game, we know the pairing equation $e(\text{COM}^*, G_2) = e(C^*, pk_{BLS})$ holds. Consequently, COM^* strictly satisfies the BLS verification equation $\text{Verify}(pk_{BLS}, C^*, \text{COM}^*) = 1$.

It's important to ensure that SIM never trivially queried the BLS oracle for the target digest C^* . By Theorem 1 of the KZG scheme [19], standard KZG polynomial commitments are binding under the q -SDH assumption. Because $\mathcal{B}^* \notin \mathcal{Q}$, the binding property guarantees that its resulting unkeyed polynomial commitment C^* is mathematically distinct from all previously queried unkeyed commitments $\{C_1, \dots, C_q\}$ with overwhelming probability. Thus, $C^* \notin \{C_i\}$, and the BLS oracle was never queried on C^* .

SIM outputs (C^*, COM^*) as its valid BLS existential forgery. We now argue that A's view in this simulated environment is mathematically identical to the real EUF_CMA game.

- (1) **Parameters and Keys:** The public parameters pp are generated using the Setup algorithm. The simulated master evaluation key $\mathcal{K}_{\text{opr}}$ is sampled uniformly at random from $\mathbb{Z}_p^*$, exactly matching an honest registration.
- (2) **Public Key:** In the real game, $pk = sk \cdot G_2$ where $sk \leftarrow_{\$} \mathbb{Z}_p^*$. In the simulation, SIM sets $pk = pk_{BLS}$. Because the BLS Challenger samples sk_{BLS} uniformly at random from $\mathbb{Z}_p^*$, the

simulated public key is uniformly distributed in $\mathbb{G}_2$, perfectly matching the real-world distribution.

- (3) **Registration Oracle:** In the real world, a keyed commitment COM_{sk} and applies its secret key to output $sk \cdot C$. In the simulation, SIM computes the exact same authentic digest C (since SIM possesses the true K_{prf} hence can compute all the correct tags, which ensures the exactness of the C) and queries the BLS oracle. Because the BLS signing algorithm computes $\sigma_i = sk_{\text{BLS}} \cdot C$, the simulated Keyed Commitment COM_{sk}^i is the exact mathematical scalar multiplication that an honest enclave would produce.

Because SIM's simulation is perfect, SIM's probability of forging a BLS signature is exactly equal to A's advantage $\text{Adv}_A^{\text{EUF-CMA}}(\lambda)$.

Because the BLS signature scheme is proven to be EUF-CMA secure assuming the hardness of the co-Computational Diffie-Hellman

(co-CDH) problem across asymmetric bilinear groups, the success probability of SIM must be strictly bounded by a negligible function $\text{negl}(\lambda)$. Therefore, the data forgery advantage $\text{Adv}_A^{\text{EUF-CMA}}(\lambda)$ must also be negligible. $\square$

Artifacts for Result Replication

To replicate our experiment, one should first install all the required packages following the `requirements.txt`'s instruction. Then one should run through the file `Double-Blind.ipynb` to achieve the following

- Run a toy example on COCO dataset (section 1)
- Obtain the static-size performance (section 2)

However, a reader can also obtain our full experiment result on more parameters within this repository. One should locate the file `Formal_benchmark_results.csv` in the repository.